

PROJECT EXECUTION & BUILD PLAN

CampusOS

From Zero to Pilot Launch — Phases, Tools, and Milestones

Document Version: 1.0

Companion Document to: CampusOS SRS v1.0

Date: July 2026

Table of Contents

1. Executive Summary
2. Technology Stack
3. Team Roles & Working Model
4. Environment & Tooling Setup Checklist
5. Phase 0: Planning & Design
6. Phase 1: Foundations
7. Phase 2: The Utility Core
8. Phase 3: The Engagement Engine
9. Phase 4: Hardening, Testing & Launch
10. Database Build Order & Migration Plan
11. Testing Strategy
12. Deployment & DevOps
13. Risk Register
14. Post-Launch Roadmap (v2+)

1. Executive Summary

This document translates the CampusOS SRS into an actionable, phased build plan. It sequences work so that value ships early (Marketplace and Lost & Found before the more complex Skill Barter economy), keeps infrastructure overhead low for a small/student team (Supabase + Next.js + managed hosting), and builds in explicit checkpoints for security (RLS), data integrity, and abuse-resistance before each phase is considered "done."

The plan is organized into five phases: Phase 0 (Planning & Design), Phase 1 (Foundations), Phase 2 (Utility Core), Phase 3 (Engagement Engine), and Phase 4 (Hardening & Launch). Each phase lists concrete tasks, deliverables, the tools used, and acceptance criteria, so progress is verifiable rather than aspirational.

2. Technology Stack

Layer	Technology	Purpose / Justification
Frontend Framework	Next.js 15 (App Router)	Server components for fast, SEO-friendly pages; file-based routing maps cleanly to CampusOS's module structure.
Styling	Tailwind CSS	Rapid, consistent UI development; pairs well with a component library for design-system discipline.
Language	TypeScript	End-to-end type safety across API routes, Supabase queries, and React components.
Backend / DB	Supabase (PostgreSQL)	Managed Auth, Realtime, Storage, and relational database with Row Level Security — minimal DevOps for a small team.
Authentication	Supabase Auth (email/OTP + domain allow-list)	Native support for magic links/OTP; domain restriction enforced at signup API layer.
Realtime	Supabase Realtime (WebSockets)	Powers department chat channels and live notification delivery.
File Storage	Supabase Storage	Private buckets with signed URLs for resources, listing images, and attachments.
AI Features	Google Gemini API	Optional coaching agent and smart-match summary copy; called server-side only.
Hosting	Netlify (primary) / Azure Static Web Apps (alternative)	Zero-config CI/CD, preview deployments per PR, generous free tier for a pilot.
Transactional Email	Resend or Supabase SMTP integration	OTP delivery, approval notices, digest emails triggered via Database Webhooks.
Version Control	GitHub	Source control, PR review workflow, GitHub Actions for CI.
Design Prototyping	Figma or HTML "stitch" mockups	Visual prototypes converted into production TypeScript components (established workflow).
Testing	Vitest/Jest (unit), Playwright (E2E)	Automated regression coverage across modules before each phase gate.

3. Team Roles & Working Model

Even a small or solo team benefits from explicitly rotating through these functional "hats" rather than context-switching randomly. If the team grows, these map directly onto separate contributors.

Role / Hat	Responsibility
Product / Scope Owner	Maintains the SRS, decides what's in/out of each phase, liaises with the faculty/Council sponsor.
Backend & Schema Owner	Supabase schema, RLS policies, triggers, migrations, escrow/reputation logic.
Frontend Owner	Next.js routes, components, Tailwind design system, responsiveness.
Integrations Owner	Auth flows, Storage, Realtime, Webhooks, Gemini API, email provider.
QA / Release Owner	Test plans, Playwright suites, staging verification, go/no-go per phase gate.

3.1 Confirm-Before-Code Workflow

For every non-trivial feature, the plan is written and explicitly approved before implementation begins — matching the established preference for a confirm-before-code workflow. Each phase below is designed to be broken into these approve-then-build increments.

4. Environment & Tooling Setup Checklist

- Create the Supabase project; note the project URL and anon/service keys; restrict the service key to server-only environment variables.
- Initialize the Next.js 15 project with the App Router and TypeScript template; configure Tailwind CSS.
- Set up a GitHub repository with a protected main branch and PR-based workflow.
- Connect the repository to Netlify (or Azure Static Web Apps) for automatic preview deployments per branch/PR.
- Configure environment variables (Supabase URL/keys, Gemini API key, email provider key) separately for local, preview, and production environments — never commit secrets.
- Install and configure the Supabase CLI locally for schema migrations and type generation (supabase gen types typescript).
- Set up GitHub Actions (or Netlify build hooks) to run lint, type-check, and unit tests on every PR.

5. Phase 0: Planning & Design

Goal: de-risk the build before writing production code. Duration guide: 1–2 weeks.

Task	Deliverable	Tools
Finalize SRS sign-off with faculty/Council sponsor	Approved SRS + list of open questions resolved (Section 8.4 of SRS)	Google Docs / this SRS
Design core screens as HTML/Figma "stitch" mockups (Home feed, Marketplace listing, Calendar, Barter booking)	Click-through prototype covering the 6 core modules	Figma or HTML mockups
Draft the full ERD from Section 4 of the SRS	Entity-relationship diagram + column-level data dictionary	dbdiagram.io / Excalidraw
Define permission-tier thresholds and reputation weights (Section 8.1/8.2 of SRS) as a config table draft	Config table schema + initial values	Spreadsheet → migration
Confirm domain allow-list and legal/consent copy (DPDP Act alignment)	Signup consent text + domain whitelist config	Doc + .env config

Acceptance Criteria: mockups reviewed and approved; ERD reviewed against every FR in the SRS to confirm no entity is missing; sponsor has signed off on moderation authority (who approves Calendar events and resolves Reports).

6. Phase 1: Foundations

Goal: a verified user can sign up, log in, and land on a personalized (empty) dashboard. Duration guide: 2–3 weeks.

6.1 Tasks

Task	Deliverable	Tools
Set up Next.js 15 + Supabase project scaffolding	Deployed "hello world" on Netlify preview URL	Next.js, Supabase, Netlify
Implement domain-locked email/OTP signup (AUTH-FR-001, AUTH-FR-002)	Working signup/verify flow rejecting non-institutional domains	Supabase Auth
Build the public.profiles auto-sync trigger (AUTH-FR-004)	SQL trigger + migration, tested against auth.users insert	Supabase SQL / CLI migrations
Implement session login, JWT handling, password reset (AUTH-FR-003, AUTH-FR-005)	Login/reset flows with Next.js middleware guarding protected routes	Supabase Auth, Next.js middleware
Build base RLS policy set: default-deny on all new tables, self-row policies on profiles	RLS policy migration file + policy test notes	Supabase RLS, SQL
Scaffold the app shell: navigation, empty dashboard, profile settings page	Navigable shell reflecting all 6 modules (even if module pages are placeholders)	Next.js App Router, Tailwind
Stand up the Reputation & Permission Engine tables and initial scoring trigger stub (REP-FR-001, REP-FR-002)	reputation_events table + profiles.reputation_score/permission_tier columns wired	Supabase SQL

6.2 Phase Gate — Acceptance Criteria

- A student with a valid institutional email can sign up, verify via OTP, and is rejected if using a personal email domain.
- On first login, a public.profiles row exists automatically with department/year captured and permission_tier = 'Reader'.
- Attempting to query another user's private data directly via the Supabase client is blocked by RLS (manually verified with a second test account).

7. Phase 2: The Utility Core

Goal: ship the modules with the fastest path to individually-felt value, per the Cold-Start guardrail (SRS Section 7.1).
Duration guide: 3–4 weeks.

7.1 Tasks — Marketplace & Lost & Found

Task	Deliverable	Tools
Build listings CRUD with image upload (MKT-FR-001) gated at Trader tier	Create/edit/close listing flows + Storage-backed image upload	Next.js, Supabase Storage
Build search/filter UI (MKT-FR-002)	Filterable, paginated listings grid	Next.js, Postgres full-text/ILIKE search
Build Lost & Found submission + status tracking, open at Reader tier (LF-FR-001, LF-FR-002)	Lost/Found report form + Active/Found status toggle	Next.js, Supabase
Implement Report Listing/User safety flow + auto-hide at 3+ open reports (SAFE-FR-001)	Report modal + reports table + auto-hide trigger	Supabase SQL trigger
Implement transaction logging on "mark as sold" (SAFE-FR-002, MKT-FR-004)	transactions table populated on deal completion	Supabase SQL

7.2 Tasks — Admin-Approved Calendar

Task	Deliverable	Tools
Build event submission form gated at Contributor tier (CAL-FR-001)	Submission flow writing to 'pending' status	Next.js, Supabase
Build Council/Admin approval queue (CAL-FR-002)	Approve/reject-with-reason moderation screen	Next.js, Supabase
Build personalized calendar view filtered by dept/year (CAL-FR-003)	Calendar UI defaulting to viewer's department	Next.js, Tailwind

7.3 Phase Gate — Acceptance Criteria

- A Reader-tier account can submit a Lost & Found report but is correctly blocked (with a clear message) from creating a Marketplace listing.
- A submitted Calendar event is invisible campus-wide until a Council/Admin account approves it.
- Three reports against the same listing auto-hides it from public search without manual intervention.

8. Phase 3: The Engagement Engine

Goal: the modules that create daily-active habits and network effects — community, ratings, barter, and recruitment.
Duration guide: 4–5 weeks.

8.1 Tasks — Dept Communities & Universal Rating/Review Schema

Task	Deliverable	Tools
Build department Realtime chat channels + auto-enrolment (COMM-FR-001, COMM-FR-002)	Live chat UI scoped to the viewer's department	Supabase Realtime, Next.js
Build resource upload/rating for notes & PYQs (RES-FR-001, RES-FR-002)	Upload flow + signed-URL download + rating hook	Supabase Storage, Postgres
Build the universal polymorphic Interactions Engine (INT-FR-001 to INT-FR-004)	Single interactions table/API serving resources, events, listings, barter, profiles	Supabase SQL, Next.js API routes
Wire reputation_score updates to interaction events (REP-FR-001 completion)	Trigger/function recalculating score on rating/report events	Supabase SQL functions/triggers
Build "Top Rated" surfacing via scheduled materialized view refresh (SENT-FR-002)	Nightly refresh job + Top Rated widget	Supabase scheduled functions / pg_cron

8.2 Tasks — Skill Barter (Time-Credit Economy)

Task	Deliverable	Tools
Build skill offer creation + browsing (BAR-FR-001)	Offer listing + tag-based browse UI	Next.js, Supabase
Build session booking with starter-credit allowance (BAR-FR-002)	Booking flow enforcing credit-balance checks	Supabase SQL
Implement escrow-style hold logic on booking (BAR-FR-003)	escrow_transactions table + hold/release/reverse state machine	Supabase SQL functions
Implement dual-confirmation completion flow (BAR-FR-004)	Confirm-completion UI for both parties + auto-release logic	Next.js, Supabase
Implement dispute/cancellation handling (BAR-FR-005)	Dispute flag + Admin review queue integration	Next.js, Supabase
Build the credit ledger view (BAR-FR-006)	Append-only ledger UI with running balance	Next.js, Supabase

8.3 Tasks — Recruitment Portal & Notifications

Task	Deliverable	Tools
Build opening creation + skill-tag matching (REC-FR-001, REC-FR-002)	Posting form + tag-intersection matcher job	Next.js, Supabase
Build application + accept/reject + auto-	Applicant ranking view + acceptance-	Next.js, Supabase Realtime

Task	Deliverable	Tools
created chat thread (REC-FR-003, REC-FR-004)	triggered chat	
Build the Notification System via Database Webhooks (NOTIF-FR-001, NOTIF-FR-002)	In-app notification center + email digests for Skill Match / Instant Match	Supabase Webhooks, Resend/SMTP

8.4 Phase Gate — Acceptance Criteria

- A completed barter session correctly moves credits from held to released only after both parties confirm, and reverses cleanly on a raised dispute.
- A new Recruitment opening with matching skill tags generates a notification for a test account with those tags within seconds of posting.
- Reputation score visibly changes after a rating event, and a demoted score correctly removes Trader-tier access on next permission check.

9. Phase 4: Hardening, Testing & Launch

Goal: move from "feature-complete" to "trustworthy enough for real students and real transactions." Duration guide: 2–3 weeks.

Task	Deliverable	Tools
Full RLS audit — attempt cross-user reads/writes on every table from a non-owner test account	RLS audit checklist with pass/fail per table	Supabase Studio, manual + scripted tests
Load-test key indexed queries (dashboard, feed, ledger) against seeded data at pilot scale	Performance report against Section 6.1/6.4 NFR targets	k6 or Artillery
Run the full Playwright E2E suite across all six modules	Green E2E run in CI on the release branch	Playwright, GitHub Actions
Security pass: rate limiting on auth/report endpoints, file-type validation on uploads	Verified rate-limit and upload-validation behaviour	Next.js middleware, Supabase Storage policies
Legal/consent review: DPDP-aligned consent copy, data-deletion request path	Published privacy notice + deletion request flow	Docs + Next.js settings page
Pilot onboarding: seed one department, recruit first Contributors/Traders manually to avoid cold-start dead-air	Onboarded pilot cohort + seeded initial listings/resources	Manual outreach, admin tools
Production deploy + monitoring setup	Live production URL + uptime/error monitoring dashboard	Netlify/Azure, Supabase logs, (optional) Sentry

9.1 Launch Gate — Go/No-Go Checklist

- All Phase 1–3 acceptance criteria remain green after the hardening pass.
- No table lacks an explicit RLS policy (default-deny verified, not assumed).
- Sponsor/Admin account access confirmed and moderation queue tested end-to-end with a real report.
- Rollback plan documented (previous Netlify/Azure deployment can be restored in under 5 minutes).

10. Database Build Order & Migration Plan

Tables should be created in dependency order so foreign keys always reference an existing parent table.

Recommended migration sequence:

- 1. profiles (depends on auth.users via trigger) + reputation_events + a config table for tier thresholds.
- 2. calendar_events, channels, channel_messages, resources — all depend only on profiles.
- 3. listings, lost_found_items, reports, transactions — Marketplace/safety layer, depends on profiles (+ reports references any entity generically).
- 4. credit_ledger, barter_sessions, escrow_transactions — depends on profiles; escrow_transactions depends on barter_sessions.
- 5. openings, applications — depends on profiles.
- 6. interactions, notifications — built last since they reference multiple prior entity types polymorphically.

Each migration should be written as a versioned SQL file via the Supabase CLI (supabase migration new <name>) and applied through CI on merge to main, never applied ad hoc against production.

11. Testing Strategy

Test Layer	Scope	Tooling
Unit	Reputation formula, permission-tier gating logic, escrow state transitions	Vitest/Jest
Integration	API route ↔ Supabase interaction, RLS policy enforcement per role	Vitest + Supabase local dev stack
End-to-End	Full user journeys: signup → list item → report → resolve; book barter → dual-confirm → ledger update	Playwright
Security	RLS bypass attempts, rate-limit verification, file-upload validation	Manual + scripted (Section 9 hardening pass)
Load	Feed/dashboard/ledger queries at simulated pilot-scale concurrency	k6 or Artillery

12. Deployment & DevOps

- Environments: local (Supabase local dev + Next.js dev server), preview (per-PR Netlify deploy against a Supabase staging project), production (main branch → Supabase production project).
- CI: GitHub Actions runs lint, type-check, unit tests, and the Playwright suite on every PR; merge to main is blocked on a green run.
- Migrations applied via Supabase CLI as part of the deploy pipeline, never manually against production.
- Secrets (service role key, Gemini key, email provider key) stored in Netlify/Azure environment variable settings, never committed.
- Monitoring: Supabase's built-in logs/metrics for database and Auth; optional Sentry integration for frontend error tracking post-launch.

13. Risk Register

Risk	Likelihood	Impact	Mitigation
Cold start — too few users for network-effect modules	High	High	Sequence launch with Marketplace/Lost & Found first (Section 7.1 of SRS); manually seed pilot content.
RLS misconfiguration exposing private data	Medium	High	Default-deny policy discipline + Phase 4 full RLS audit before launch.
Escrow logic bug causing credit loss/duplication	Medium	Medium	Dedicated unit tests on every escrow state transition; append-only ledger for auditability.
Institutional pushback on peer Marketplace transactions	Low-Medium	Medium	Secure a faculty/Council sponsor early (Phase 0) and align consent/legal copy.
Supabase free/pro tier limits hit at scale	Low (pilot), Medium (full campus)	Medium	Index-first schema design (Section 4.1 of SRS); monitor usage and upgrade plan proactively.
Small team bandwidth / scope creep	High	Medium	Hold firmly to the phase gates in this document; defer roadmap items (Section 14) explicitly.

14. Post-Launch Roadmap (v2+)

- Weighted skill-matching model for the Recruitment Portal (beyond exact tag intersection).
- Native mobile wrapper (Capacitor/Expo) once PWA usage data justifies the investment.
- Live campus activity heatmap via optional IoT/Wi-Fi density integration.
- Multi-campus federation with an institution_id namespace, enabling the model to expand beyond a single college.
- Tunable reputation-weight experimentation once real pilot behaviour data is available (Appendix 8.1 of SRS).